

[Open in app](#)

Pune AI Community · Following

Text vs Graph Transformer

Step by step deep dive into the comparison

10 min read · Just now



Yogesh Haribhau Kulkarni (PhD)

 Listen

 Share

 More

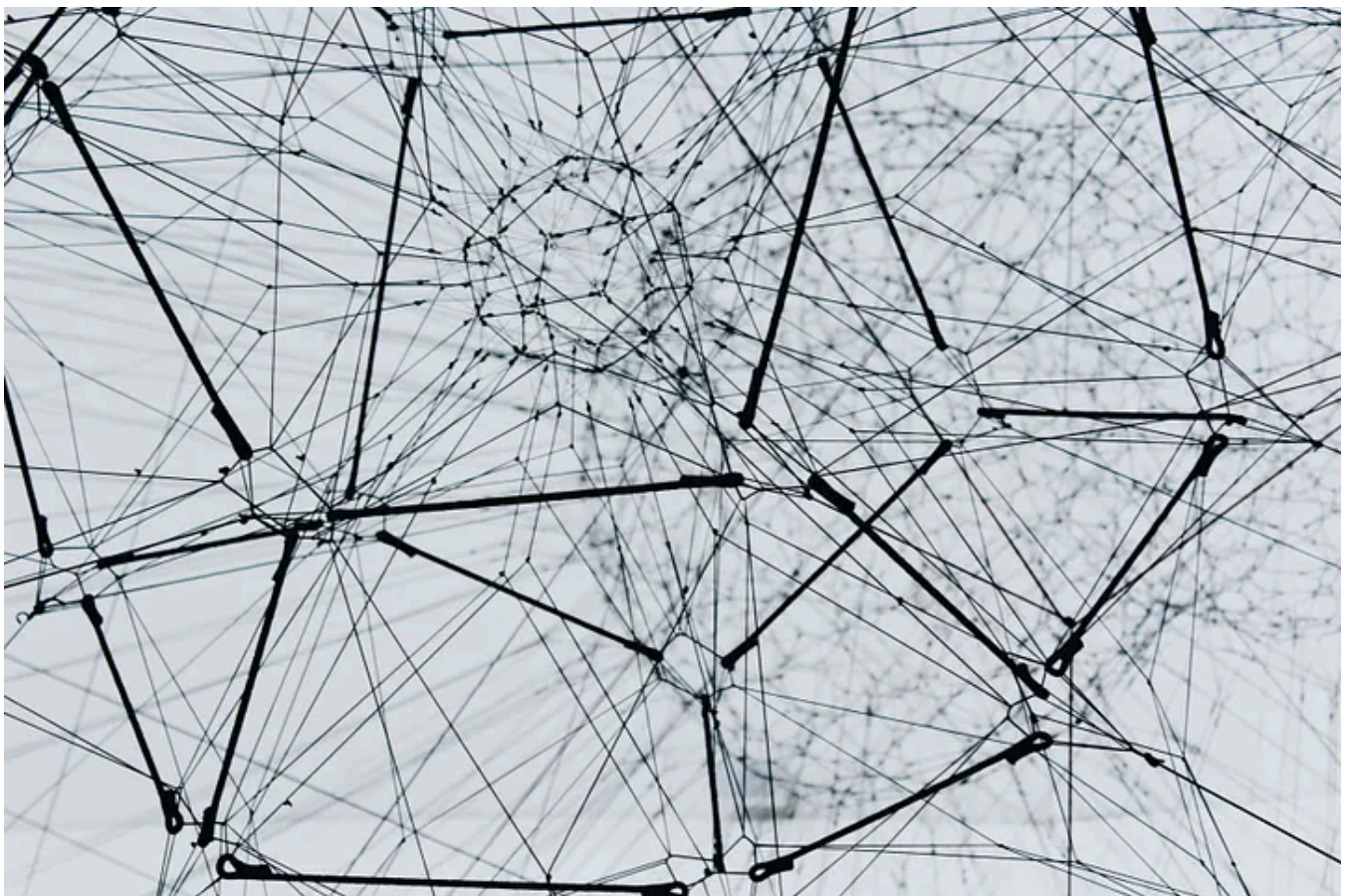
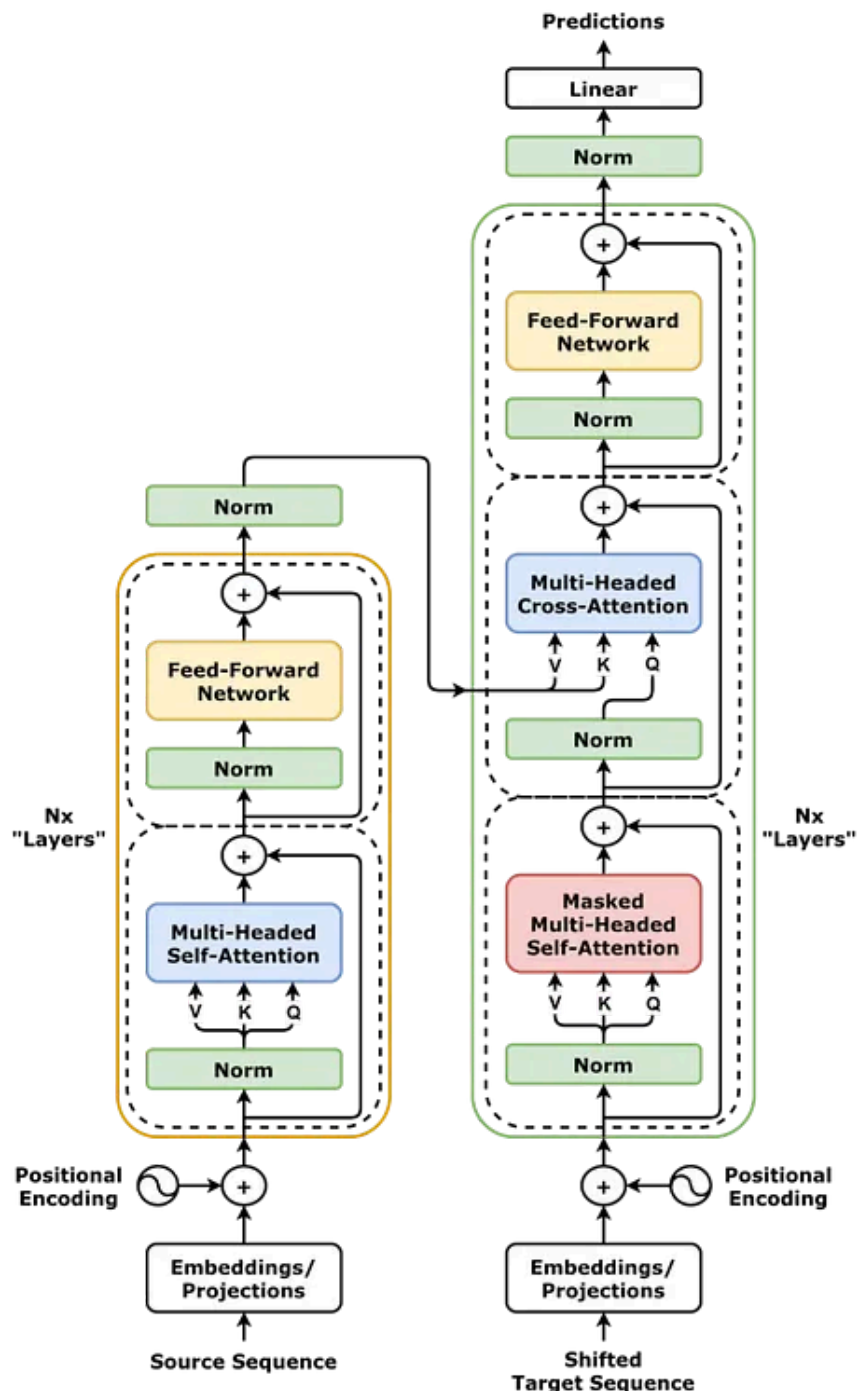


Photo by [Alina Grubnyak](#) on [Unsplash](#)

If you have spent any time in Generative AI (Artificial Intelligence), I am sure you have heard of Transformer architecture. The one that powers, BERT, GPT, Gemini, etc. They all run on the same engine: self-attention over a sequence of tokens.



(Source: [Wikimedia](#))

The architecture is so successful that researchers have been asking a natural question for the last few years: can we apply the same ideas to graphs?

By the end, you will understand where Graph Transformers work the same way, where they diverge, and why the divergences are not arbitrary engineering choices but mathematically necessary responses to a different kind of data.

First, Why Graphs at All?

Sequences are data with one kind of structure: order. Word 1 comes before word 2. Pixel row 47 comes before row 48. The Transformer was built to model this structure

using attention, allowing every element to look at every other element and decide how much to “attend” (or focus on) to it.

Graphs are data with a different kind of structure: arbitrary connectivity. A molecule has atoms (nodes) connected by bonds (edges) in no particular order. A social network has people connected by relationships.

None of these have a natural “first” element. None sit on a line. The Transformer’s positional encoding, which assumes a line, breaks immediately. The rest of the architecture needs rethinking too.

Let us go component by component.

1. Input Embeddings

Text Transformer

In a text Transformer, the input is a sequence of tokens, words or subwords from a fixed vocabulary (typically 30,000 to 50,000 items). Each token is mapped to a dense vector through a **learned embedding table**: a matrix of shape $[\text{vocab_size} \times d_model]$ where d_model is the embedding dimension (512 in the original Transformer, 768 in BERT-base).

Token: "cat" → index 2743 → embedding vector of shape [768]

Every token starts as a single vector. The sequence of N tokens becomes a matrix of shape $[N \times d_model]$. That matrix is what flows into the Transformer layers.

Crucially, at this stage, **all tokens are treated equally**. The embedding table knows nothing about order or context. A standalone “bank” and “bank” in “river bank” start as the same vector, context will differentiate them through attention, not through the embeddings themselves.

Graph Transformer

A graph has two kinds of input: **nodes** and **edges**, each potentially carrying features. **Node features** are analogous to token embeddings. A node might represent:

- An atom in a molecule → features: atomic number, charge, aromaticity

- A logic gate in a netlist → features: gate type (AND/OR/NOT), fanin, fanout
- A room in a floorplan → features: area, aspect ratio, room type

If the features are categorical (like gate type), they are embedded through a lookup table exactly like token embeddings. If continuous (like area), they are projected through a linear layer into `d_model` dimensions.

Edge features have no analogue in text Transformers. An edge might carry: bond type (single/double/aromatic in molecules), wire delay (in netlists), wall thickness (in floorplans). Edge features are embedded separately and injected into the attention computation, more on this below.

```
Node v: [gate_type=AND, fanin=2, fanout=3] → linear projection → vector [d_model]
Edge (u,v): [wire_delay=0.3ns, wire_length=12μm] → linear projection → vector [d_model]
```

So the input to a Graph Transformer is not a single matrix but a **triple**: node feature matrix X , edge feature matrix E , and adjacency structure A .

2. Positional Encodings

This is where the architectures diverge most dramatically, and where Graph Transformers face their hardest unsolved problem.

Text Transformer

Position is obvious in a sequence: token 1, token 2, ..., token N. The original Transformer adds a sinusoidal signal to each token embedding:

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

This gives each position a unique fingerprint. Token at position 5 gets a different additive signal than token at position 47. The model can use this to understand distance and order.

BERT and its descendants learned these positional embeddings instead of computing them analytically, but the concept is the same: each integer position maps to a unique vector.

Graph Transformer

A graph has no canonical ordering of nodes. Node “G42” in a netlist has no natural position 1, 2, or 47. If you arbitrarily assigned positions, your model would learn spurious correlations based on the arbitrary ordering. Two structurally identical graphs with differently-numbered nodes would look different to the model, clearly wrong.

Graph Transformers use **structural positional encodings** that describe each node’s position in terms of the graph’s own topology. The main approaches:

Laplacian Eigenvector PE (LapPE): The graph Laplacian $L = D - A$ (where D is the degree matrix and A is the adjacency matrix) has eigenvectors that encode global structural position. Nodes that play similar roles in the graph get similar eigenvectors. Used in models like Graphormer and SAN.

The catch: eigenvectors have **sign ambiguity**. The same graph can produce eigenvectors flipped in sign, making training unstable. This is an active research problem solutions include random sign-flipping during training (augmentation) and learned sign-invariant networks like SignNet.

Random Walk Structural Encoding (RWSE): For each node v , compute the probability of a random walk starting at v returning to v after k steps, for $k = 1, 2, \dots, K$. This gives a K -dimensional fingerprint of v ’s local neighbourhood structure. Stable, no sign ambiguity but captures only local structure, not global position.

Distance Encoding: Encode the shortest-path distance between every pair of nodes as a bias added to the attention logits. Graphormer does this. Captures global structure but costs $O(n^2)$ to compute.

The geometric case (relevant to CAD, molecular graphs, floorplans): nodes have actual coordinates in 2D or 3D space. You might think: just use the coordinates as positional encoding. The problem is that coordinates are not **invariant** to rotation, translation, or reflection. A molecule rotated 90 degrees is the same molecule. A floorplan mirrored is the same floorplan. Positional encodings derived from raw

coordinates would make the model think they are different, a serious error.

Equivariant architectures (EGNN, SE(3)-Transformers) handle this by ensuring that the model's outputs transform correctly when the input geometry transforms. This is the frontier of geometric deep learning.

3. Self-Attention

Text Transformer

Self-attention is the heart of the Transformer. For a sequence of N tokens, each represented as a vector in $\mathbb{R}^{d_{\text{model}}}$, self-attention computes three projections for every token:

```
Query:  Q = X · W_Q      shape [N × d_k]
Key:    K = X · W_K      shape [N × d_k]
Value:  V = X · W_V      shape [N × d_v]
```

Attention scores are computed between every pair of tokens:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

This is **dense**, every token attends to every other token. For a sequence of length N , the attention matrix is $N \times N$. This is computationally expensive (quadratic in N) but expressively powerful: information can flow between any two tokens in a single layer.

Graph Transformer

The key design decision in Graph Transformers: **should attention be dense (all nodes attend to all nodes) or sparse (nodes only attend to their neighbours)?**

Full attention (like text Transformers): Every node attends to every other node. The graph structure is not used to restrict attention, instead, it is injected as a *bias* into the attention scores. Graphormer does this:

$$\text{Attention_score}(i,j) = (Q_i \cdot K_j^T) / \sqrt{d_k} + \text{bias}(i,j)$$

where $\text{bias}(i,j)$ encodes the shortest-path distance between nodes i and j (or an edge feature if i and j are directly connected). This way the graph structure informs attention without hard-constraining it.

Sparse attention (neighbourhood-only): Nodes only attend to their direct neighbours, the edges in the graph define the attention pattern. This is equivalent to Graph Attention Networks (GAT), an important predecessor to Graph Transformers. It is $O(|E|)$ instead of $O(N^2)$, which matters for large graphs like full chip netlists with millions of gates.

Hybrid approaches: The GPS (General, Powerful, Scalable) Graph Transformer combines both: local message passing (sparse, over neighbours) and global attention (dense, over all nodes) in parallel, then concatenates the results. This gets the best of both worlds at manageable cost.

Edge features in attention: When edges carry features, they modify the attention computation:

$$\text{Attention_score}(i,j) = (Q_i \cdot K_j^T + Q_i \cdot E_{ij}^T) / \sqrt{d_k}$$

where E_{ij} is the projected edge feature between nodes i and j . The value aggregation can similarly be modulated by edge features. This has no analogue in text Transformers because text sequences have no explicit edge features.

4. Cross-Attention

Text Transformer

Cross-attention is used in encoder-decoder architectures (the original Transformer, T5). The decoder generates Queries from its own state; the Keys and Values come from the encoder's output:

```
Q = decoder_state · W_Q  
K = encoder_output · W_K  
V = encoder_output · W_V
```

This allows the decoder to “look at” the encoded source sequence while generating each output token, essential for tasks like translation.

Graph Transformer

Cross-attention in Graph Transformers arises in **graph-to-graph** or **graph-to-sequence** settings:

- **Graph-to-sequence** (e.g., molecule captioning, netlist description): encoder processes the graph; decoder generates text. Cross-attention works exactly as in text Transformers, decoder Queries attend to graph node encodings.
- **Graph-to-graph** (e.g., midcurve extraction, netlist simplification): one graph is encoded; the decoder generates a new graph. Cross-attention allows the output graph's nodes to attend to the input graph's nodes during generation. This is structurally identical to seq2seq cross-attention but over node sets rather than token sequences.
- **Multimodal settings** (e.g., geometry + language): a text description of a CAD part cross-attends to a graph representation of the same part's geometry. The text tokens are Queries; the graph nodes provide Keys and Values. This is an active research direction in geometric foundation models.

5. Encoder-Only, Decoder-Only, Encoder-Decoder: Both Worlds

Text Transformer

The three main architectural families and their applications:

Architecture Examples:

- Encoder-only BERT, RoBERTa: Classification, NER, QA
- Decoder-only GPT-2, GPT-4, LLaMA: Text generation
- Encoder-decoder T5, BART: Translation, summarisation

Encoder-only models process the full input bidirectionally, every token attends to every other token. Decoder-only models use causal (masked) attention, each token attends only to previous tokens, enabling autoregressive generation. Encoder-decoder models combine both.

Graph Transformer

The same three families exist, mapped to graph tasks:

Encoder-only (graph understanding): The most common case. The full graph is processed bidirectionally, every node attends to every other node (or neighbour), building rich node and graph representations. Used for:

- Molecular property prediction (is this drug candidate toxic?)
- Chip netlist analysis (which gates are on the critical timing path?)
- Social network node classification (what community does this user belong to?)
- CAD feature recognition (is this a hole, a fillet, a boss?)

Models: Graphormer, GPS, SAN, GIT (Graph Inductive Transformer).

Decoder-only (graph generation, autoregressive): Generate a graph node by node, edge by edge, with each step attending only to previously generated elements. Used for:

- Drug molecule design (generate a valid molecular graph with target properties)
- Logic synthesis (generate an optimised gate-level netlist from a specification)
- Scene graph generation

The ordering problem is acute here: in what order do you generate nodes? BFS order? Random? The choice affects quality, and there is no canonical answer for most graph types. This is one reason autoregressive graph generation is harder than autoregressive text generation.

Encoder-decoder (graph-to-graph or graph-to-sequence): Encode a source graph; decode a target graph or sequence. Used for:

- Midcurve extraction: encode a 2D B-Rep Profile graph, decode a 1D midcurve graph
- Netlist simplification: encode a detailed gate-level netlist, decode an abstracted version
- Molecular retrosynthesis: encode a product molecule graph, decode reactant graphs
- Graph-to-text: encode a knowledge graph, decode a natural language description

The decoder side faces the generation ordering problem. One promising alternative: **non-autoregressive decoding**, where all output nodes and edges are predicted simultaneously, trained with an optimal transport loss that finds the best matching between predicted and target graphs without requiring a fixed node ordering.

Summary: The Key Differences at a Glance

Component	Text Transformer	Graph Transformer
Input	Token sequence, one matrix	Nodes + edges + adjacency, three tensors
Input embeddings	Lookup table (discrete vocab)	Linear projection (continuous or categorical node/edge features)
Positional encoding	Sinusoidal or learned integer-position PE	LapPE, RWSE, distance encoding, or equivariant coordinate encoding
Self-attention	Dense, all tokens attend to all tokens	Dense (full) or sparse (neighbourhood-only) or hybrid
Edge features	Not applicable	Modify attention scores and value aggregation
Cross-attention	Decoder attends to encoder output	Same, but over node sets; also used in graph-to-graph tasks
Encoder-only	BERT — classification, understanding	Graph property prediction, node classification
Decoder-only	GPT — text generation	Autoregressive graph generation (hard ordering problem)
Encoder-decoder	T5 — translation, summarisation	Graph-to-graph transforms, graph-to-text

Where the Hard Problems Live

If you want to go deeper into Graph Transformers, the open problems are:

Positional encoding on geometric graphs. How do you encode position when nodes have spatial coordinates that must be treated in a rotation-invariant way? This is the frontier of SE(3)-equivariant architectures.

Scalability. Dense attention is $O(N^2)$ in the number of nodes. For a chip with a million gates, this is not feasible. Sparse attention helps but loses the global expressiveness. Efficient approximations, analogous to Longformer and Linformer in text, are an active area.

Graph generation quality. Non-autoregressive generation is theoretically cleaner for graphs but harder to train. The optimal transport loss is expensive and the training signal is noisy. Better approaches are needed.

Benchmarking. Text Transformers have clean benchmarks (GLUE, SuperGLUE, BIG-bench). Graph Transformer benchmarks are more fragmented and often show inconsistent results across datasets. The Long Range Graph Benchmark (LRGB) is a step toward fixing this.

Where to Start

If you are an enthusiast wanting to get hands-on:

1. Implement vanilla self-attention from scratch in PyTorch: Andrej Karpathy's `nanoGPT` tutorial is the clearest path.
2. Read the GPS paper (Rampášek et al., 2022): it is the clearest synthesis of Graph Transformer design choices.
3. Install PyTorch Geometric and run its Graphormer example on the ZINC molecular dataset.
4. Look at the BahuMeetiya repository (github.com/yogeshkk/bahumeetiya) for experiments applying these ideas to 1D, 2D, and 3D geometric domains.

The Graph Transformer is not a replacement for the text Transformer. It is the same insight, attention is a powerful way to aggregate information applied to data that refuses to sit on a line.

Author's LinkedIn Profile: [yogeshkulkarni](#)

Artificial Intelligence

Transformers

Graph Algorithms

Generative Ai

Graph Neural Networks



Following

Published in Pune AI Community

11 followers · Last published just now

The Pune AI Community (PAIC) is a collaborative platform dedicated to making AI knowledge accessible from classes to masses. We bring together AI engineers, researchers, students, startups, and professionals to learn, discuss, build, and grow together.



Edit profile

Written by Yogesh Haribhau Kulkarni (PhD)

1.9K followers · 2.1K following

PhD in Geometric Modeling | Google Developer Expert (Machine Learning) | Top Writer 3x (Medium) | More at <https://www.linkedin.com/in/yogeshkulkarni/>

No responses yet



Yogesh Haribhau Kulkarni (PhD)